

Practical Work 1: TCP File Transfer

Tran Tuan Vu - 23BI14459

December 11, 2025

1 Protocol Design

In this section, we describe the protocol designed to handle the file transfer over TCP.

The protocol follows a simple "Header-Body" structure to ensure data integrity and avoid issues where the server waits indefinitely:

1. **File Size Header:** The client first sends the size of the file (in bytes) as a `long` integer.
2. **File Content Body:** Immediately after sending the size, the client sends the actual raw bytes of the file.
3. **Termination:** The server reads exactly the number of bytes specified in the header and then closes the file stream.

Figure 1: Diagram of the File Transfer Protocol Design

(Note: Replace the image file above with your own diagram showing the flow: Client → Size → Server → Data → Server)

2 System Organization

The system is organized into a Client-Server architecture:

- **Server:** Bind to a port and listen for incoming connections. It is responsible for creating the output file and writing received data.
- **Client:** Connects to the server's IP address. It is responsible for reading the local file and streaming it to the socket.

Figure 2: System Organization and Architecture

3 Implementation

Below are the core code snippets responsible for the file transfer logic.

3.1 Client Side: Sending Data

The client reads the file size using `fseek` and sends it before the data loop.

```
1 void send_file(FILE *fp, int sockfd) {
2     char data[BUFFER_SIZE] = {0};
3     long file_size;
4
5     // Calculate file size
6     fseek(fp, 0, SEEK_END);
7     file_size = ftell(fp);
8     fseek(fp, 0, SEEK_SET);
9
10    // Send file size first
11    send(sockfd, &file_size, sizeof(file_size), 0);
12
13    // Send file content
14    while (fgets(data, BUFFER_SIZE, fp) != NULL) {
15        if (send(sockfd, data, sizeof(data), 0) == -1) {
16            perror("Error sending file.");
17            exit(1);
18        }
19        bzero(data, BUFFER_SIZE);
20    }
21 }
```

Listing 1: Client sending logic

3.2 Server Side: Receiving Data

The server reads the size first, then loops until that specific amount of bytes is received.

```
1 void write_file(int sockfd) {
2     int n;
3     FILE *fp;
4     char buffer[BUFFER_SIZE];
5     long file_size;
6     long total_received = 0;
7
8     // Receive file size
9     recv(sockfd, &file_size, sizeof(file_size), 0);
10
11    fp = fopen("received_file.txt", "wb");
12
13    // Loop until all bytes are received
14    while (total_received < file_size) {
15        n = recv(sockfd, buffer, BUFFER_SIZE, 0);
16        if (n <= 0) break;
17        fwrite(buffer, 1, n, fp);
18        total_received += n;
19        bzero(buffer, BUFFER_SIZE);
20    }
21 }
```

Listing 2: Server receiving logic